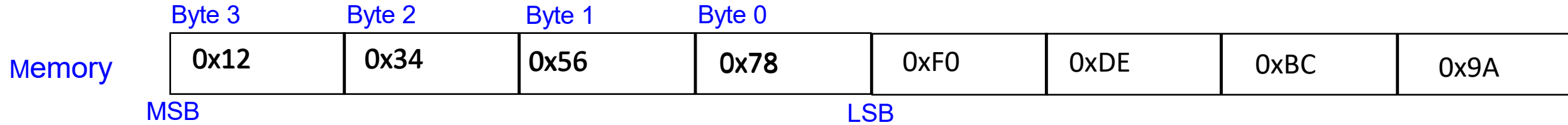


LOAD AND STORE OPERATIONS

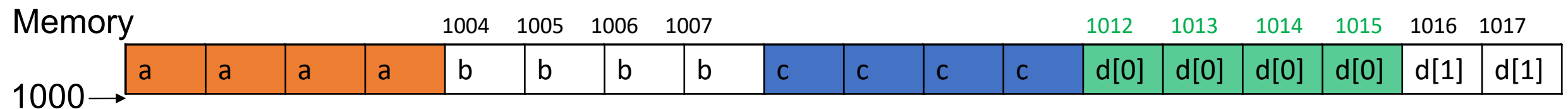
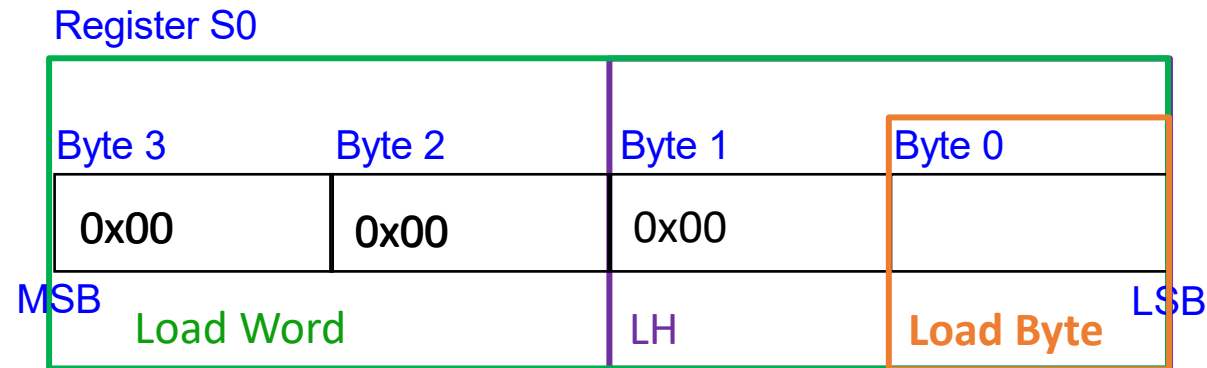


High-level code

```
int a = 0x12345678
int b = 0xF0DEBC9A
```

MIPS code

```
lb $s0, 3($gp)
lh $s0, 2($gp)
lw $s0, 0($gp)
```



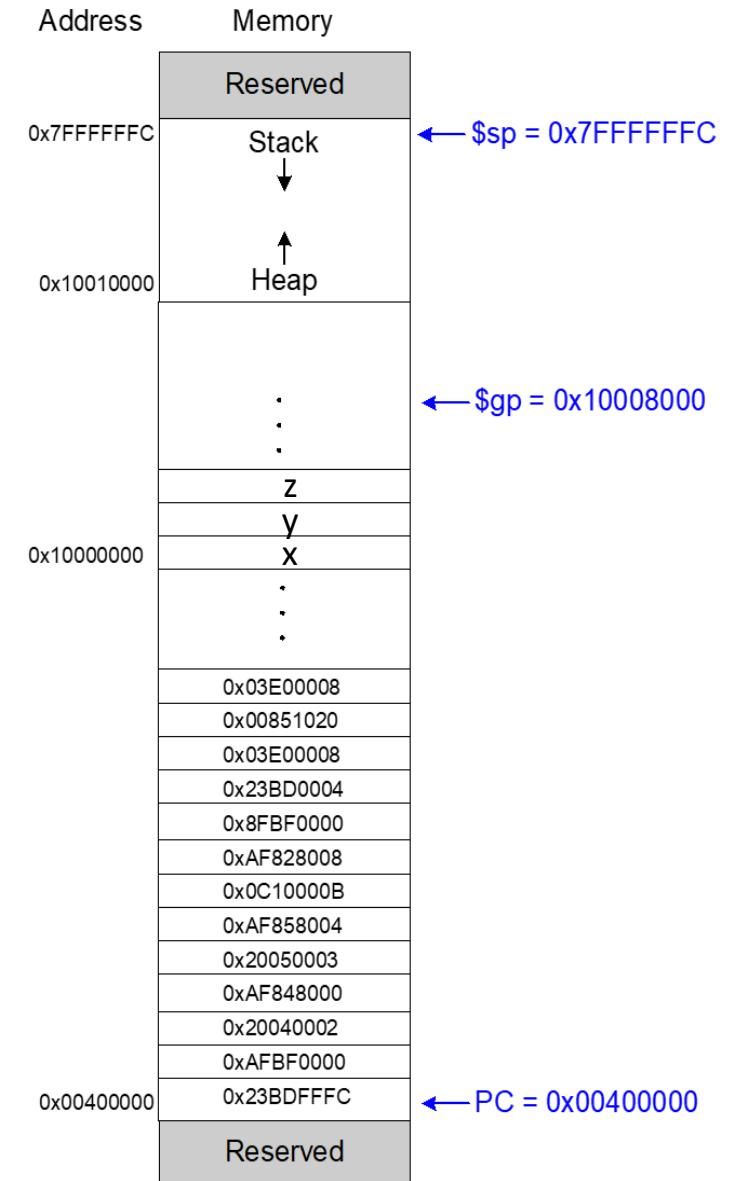
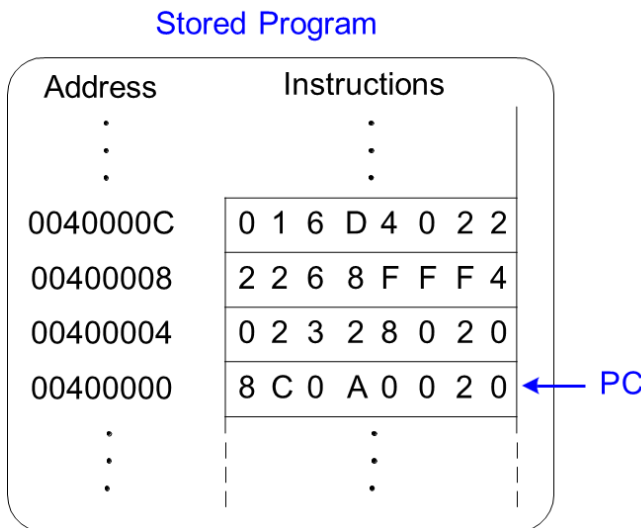
MIPS ISA – BRANCH INSTRUCTIONS

Conditional branch	branch on equal	beq \$s1,\$s2,25	if (\$s1 == \$s2) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	bne \$s1,\$s2,25	if (\$s1 != \$s2) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	slt \$s1,\$s2,\$s3	if (\$s2 < \$s3) \$s1 = 1; else \$s1 = 0	Compare less than; for beq, bne
	set on less than unsigned	sltu \$s1,\$s2,\$s3	if (\$s2 < \$s3) \$s1 = 1; else \$s1 = 0	Compare less than unsigned
	set less than immediate	slti \$s1,\$s2,20	if (\$s2 < 20) \$s1 = 1; else \$s1 = 0	Compare less than constant
	set less than immediate unsigned	sltiu \$s1,\$s2,20	if (\$s2 < 20) \$s1 = 1; else \$s1 = 0	Compare less than constant unsigned
Unconditional jump	jump	j 2500	go to 10000	Jump to target address
	jump register	jr \$ra	go to \$ra	For switch, procedure return
	jump and link	jal 2500	\$ra = PC + 4; go to 10000	For procedure call

PROGRAM COUNTER

- The processor hardware executes the program:
 - fetches (reads) the instructions from memory in sequence
 - performs the specified operation
 - continues with the next instruction
- The program counter (PC) tracks the current instruction
 - In MIPS, programs typically start at memory address 0x00400000

Assembly Code	Machine Code
lw \$t2, 32(\$0)	0x8C0A0020
add \$s0, \$s1, \$s2	0x02328020
addi \$t0, \$s3, -12	0x2268FFF4
sub \$t0, \$t3, \$t5	0x016D4022



BRANCHING INSTRUCTIONS

- Branch & Jump Instructions allow a program to execute instructions out of sequence.

- **Conditional branches**

High-level code

```
L1:  if (i == j)
      f = g + h;
L2:  else
      f = f - i;
Done: return;
```

- branch if equal: **beq**
- branch if not equal: **bne**

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h, $s3 = i, $s4 = j
      beq $s3, $s4, L1    # i!=j? L1
      sub $s0, $s0, $s3   # f = f - i
      J    Done          # jump to end of block
L1:    add $s0, $s1, $s2   # f = g + h
Done:  jr $ra            # return from routine
```

- **Unconditional branches**

- jump: **j**
- jump register: **jr**
- jump and link: **jal**

```
# $s0 = f, $s1 = g, $s2 = h, $s3 = i, $s4 = j
      bne $s3, $s4, L2    # i!=j? L2
      add $s0, $s1, $s2   # f = g + h
      J    Done          # jump to end of block
L2:    sub $s0, $s0, $s3   # f = f - i
Done:  jr $ra            # return from routine
```

Labels indicate the instruction locations in a program.

Labels cannot use reserved words and must be followed by a colon (:).

jump	j 2500	go to 10000	Jump to target address
jump register	jr \$ra	go to \$ra	For switch, procedure return
jump and link	jal 2500	\$ra = PC + 4; go to 10000	For procedure call

NO SPECIAL ISA FOR LOOPS IN ASSEMBLY!

High-level code

```
// determines the power
// of x such that 2x = 128
int pow = 1;
int x = 0;
while (pow != 128){
    pow = pow * 2;
    x = x + 1;
}
```

MIPS assembly code

```
# $s0 = pow, $s1 = x
    addi $s0, $0, 1    # Pow = 1
    add  $s1, $0, $0   # x=0
    addi $t0, $0, 128  # val=128
while: beq  $s0, $t0, done # Pow==128
        sll  $s0, $s0, 2 # Pow*=2
        addi $s1, $s1, 1 # x+=1
        j   while      # loop back to beq
done:  jr  $ra          # return
```

CONDITIONAL BRANCHING (BEQ AND BNE)

- What will be the final value at \$S1?

MIPS assembly

```
addi $s0, $0, 4
addi $s1, $0, 1
sll  $s1, $s1, 2
beq  $s0, $s1, target
addi $s1, $s1, 1
sub  $s1, $s1, $s0
```

target:

```
add  $s1, $s1, $s0
```

MIPS assembly

```
addi  $s0, $0, 4      # $s0 = 0 + 4 = 4
addi  $s1, $0, 1      # $s1 = 0 + 1 = 1
sll   $s1, $s1, 2     # $s1 = 1 << 2 = 4
bne   $s0, $s1, target # branch not taken
addi  $s1, $s1, 1     # $s1 = 4 + 1 = 5
sub   $s1, $s1, $s0   # $s1 = 5 - 4 = 1
```

target:

```
add  $s1, $s1, $s0   # $s1 = 1 + 4 = 5
```

UNCONDITIONAL BRANCHING / JUMP (J AND JR)

MIPS assembly

```
addi    $s0, $0, 4      # $s0 = 4
addi    $s1, $0, 1      # $s1 = 1
j        target         # jump to target
sra      $s1, $s1, 2     # not executed
addi     $s1, $s1, 1     # not executed
sub      $s1, $s1, $s0   # not executed
```

target:

```
add      $s1, $s1, $s0   # $s1 = 1 + 4 = 5
```

MIPS assembly

```
0x00002000    addi    $s0,    $0, 0x2010    # load 0x2010 to $s0
0x00002004    jr      $s0                  # jump to the memory
                                                address stored in $s0
0x00002008    addi    $s1,    $0, 1         # not executed
0x0000200C    sra     $s1,    $s1, 2       # not executed
0x00002010    lw      $s3,    44($s1)      # program continues
```

MIPS ARITHMETIC AND BITWISE OPERATIONS

Operation	Syntax in C	MIPS Code
AND	<code>a = b & c</code>	<code>and \$s0, \$s1, \$s2</code>
AND with a constant	<code>a = b & 0x03</code>	<code>andi \$s0, \$s1, 3</code>
OR	<code>a = b c</code>	<code>or \$s0, \$s1, \$s2</code>
OR with a constant	<code>a = b 0x03</code>	<code>ori \$s0, \$s1, 3</code>
Exclusive OR	<code>a = b ^ c</code>	<code>xor \$s0, \$s1, \$s2</code>
Exclusive OR with a const.	<code>a = b ^ 0x03</code>	<code>xori \$s0, \$s1, 3</code>

Operation	Syntax in C	MIPS Code
Shift Left	<code>a = b << 3</code>	<code>sll \$s0, \$s1, 3</code>
Right Shift (Logical)	<code>a = (uint32_t) b >> 3</code>	<code>srl \$s0, \$s1, 3</code>
Right Shift (Arithmetic)	<code>a = b >> 3</code>	<code>sra \$s0, \$s1, 3</code>
Multiplication	<code>a = b * c</code>	<code>mult \$s1, \$s2; mflo \$s0</code>
Division	<code>a = b /c</code>	<code>div \$s1, \$s2; mflo \$s0</code>
Modulo Division	<code>a = b%c</code>	<code>div \$s1, \$s2; mfhi \$s0</code>

MIPS ARITHMETIC AND BITWISE OPERATIONS

- Arithmetic Set Instructions: Test two values, place 1 (when True) and 0 otherwise.

Operation	Syntax in C	MIPS Code
SLT	$a = b < c ? 1 : 0$	slt \$s0, \$s1, \$s2
SLTU	$a = (\text{unsigned}) b < (\text{unsigned}) c ? 1 : 0$	sltu \$s0, \$s1, \$s2
SLTI	$a = b < 10 ? 1 : 0$	slti \$s0, \$s1, 10
SLTIU	$a = (\text{unsigned}) b < (\text{unsigned}) 10 ? 1 : 0$	sltiu \$s0, \$s1, 10

FUNCTIONS CALLS AND CONTROL FLOW IN MIPS

❑ The 3 steps in handling function calls:

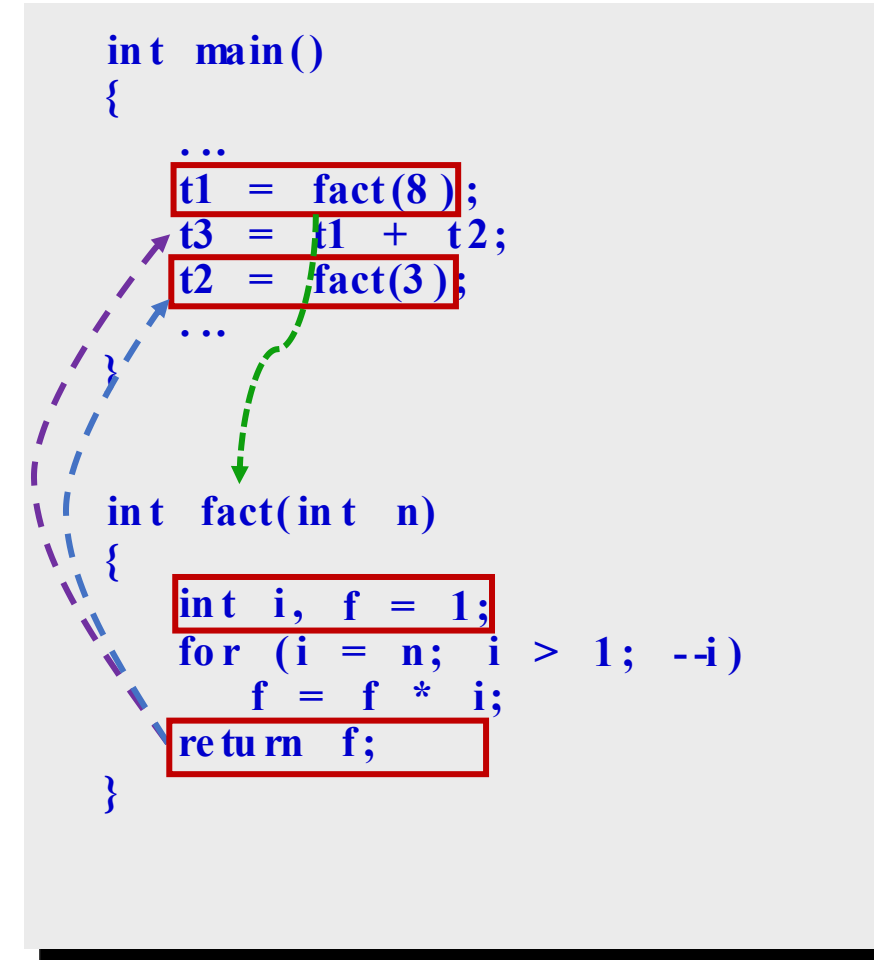
1. The program's flow of control must be changed (twice).
2. Arguments and return values are passed back and forth.
3. Local variables can be allocated and destroyed.

❑ Function call handling in MIPS:

- New instructions for calling functions.
- Conventions for sharing registers between functions.
- Use of a stack.

❑ Function invocation changes the control flow of a program

1. **Calling** the function
2. **Returning** from the function



❑ Here the **main** function calls **fact** twice, and **fact** returns to **different** locations in **main**.

❑ Each time **fact** is called, the CPU has to remember the appropriate **return address**.

❑ **main** itself is also a function! It is called by the operating system when you run the program.

CONTROL FLOW IN MIPS

❑ MIPS uses the jump-and-link instruction **jal** to call functions.

- **jal** saves the return address (the address of the *next* instruction) in the dedicated register **\$ra**, before jumping to the function.
- **jal** is the **only** MIPS instruction that can access the value of the **program counter**, so it can store the return address PC+4 in \$ra.

jal Fact

❑ To transfer control back to the caller, the function just has to jump to the address that was stored in \$ra.

jr \$ra

❑ Let's now add the jal and jr instructions that are necessary for our factorial example.

```
int main()
{
    ...
    jal Fact #t1 = fact(8)
    ...
    t3 = t1 + t2;
    ...
    jal Fact #t2 = fact(3)
    ...
}

int fact(int n)
{
    int i, f = 1;
    for (i = n; i > 1; --i)
        f = f * i;
    jr $ra;
}
```

CONTROL FLOW IN MIPS

❑ MIPS uses the jump-and-link instruction **jal** to call functions.

- **jal** saves the return address (the address of the *next* instruction) in the dedicated register **\$ra**, before jumping to the function.
- **jal** is the **only** MIPS instruction that can access the value of the **program counter**, so it can store the return address PC+4 in \$ra.

jal Fact

❑ To transfer control back to the caller, the function just has to jump to the address that was stored in \$ra.

jr \$ra

❑ Functions accept **arguments** and produce **return values**.

\$a0--\$a3 (\$4--\$7) save the arguments. (black part of the code)

\$v0--\$v1 (\$2--\$3) return the values of functions. (purple code)

❑ These conventions are not enforced by the hardware or assembler, but programmers agree to them so functions written by different people can interface with each other.

❑ How to handle additional arguments or return values?

```
int main()
{
    addi $a0,$0,8
    jal Fact #t1=fact(8)
    SW $v0, 0(t1)

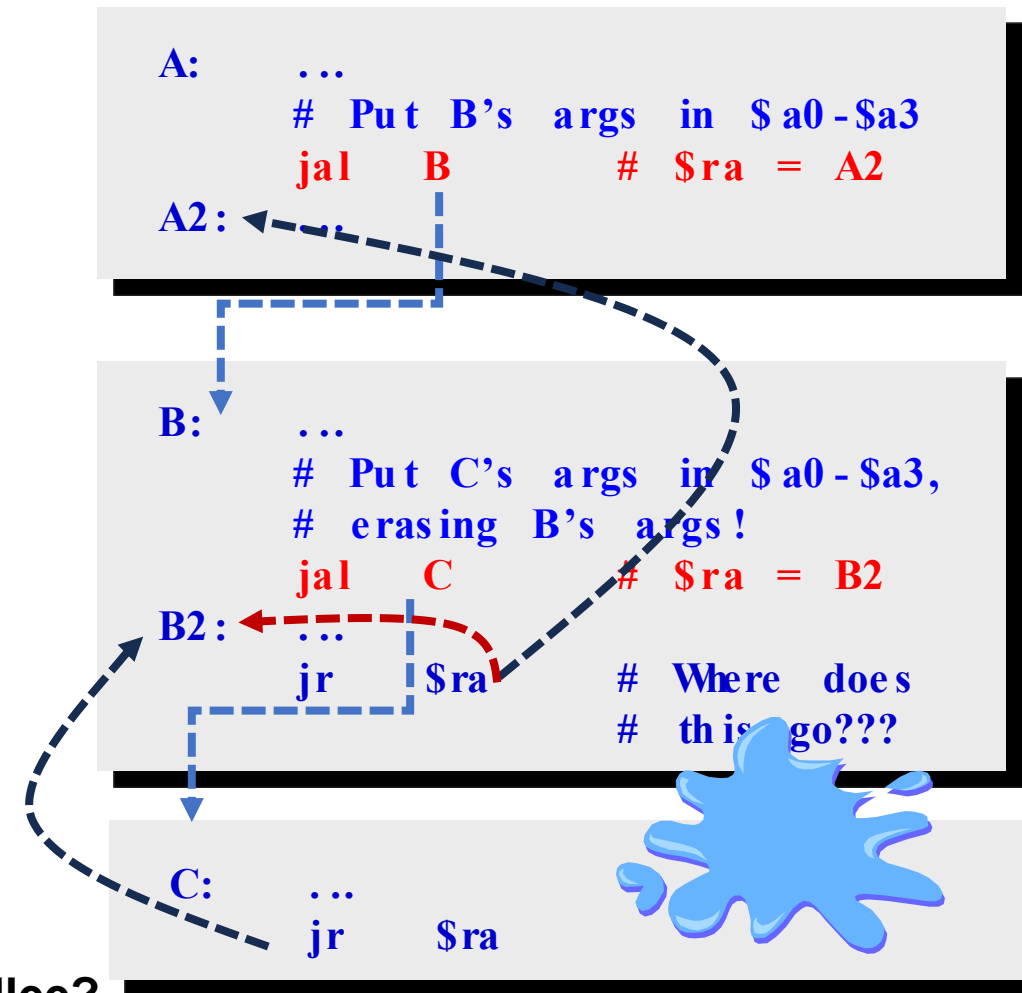
    t3 = t1 + t2;

    addi $a0,$0,3
    jal Fact #t2=fact(3)
    SW $v0, 0(t2)
}

int fact(int n)
{
    int i, f = 1;
    for (i = n; i > 1; --i)
        f = f * i;
    jr $ra;
}
```

NESTED FUNCTIONS AND REGISTER SPILLING

- ❑ What happens when you call a function that then calls another function?
- ❑ Let's say A calls B, which calls C.
 - The arguments for the call to C would be placed in **\$a0-\$a3**, thus *overwriting* the original arguments for B.
 - Similarly, **jal C** overwrites the return address that was saved in **\$ra** by the earlier **jal B**.
- ❑ The CPU has a limited number of registers for use by all functions, and it's possible that several functions will need the same registers.
- ❑ Preserve important registers from being overwritten by a function call, by **saving** them before the function executes, and **restoring** them after function completes.
- ❑ But there are two important questions.
 - Who is responsible for saving registers —the caller or callee?
 - Where exactly are the register contents saved?



WHO SAVES THE REGISTERS?

- ❑ **In the typical “black box” programming approach, the caller and callee do not know anything about each other’s implementation.**
 - Different functions may be written by different people or companies.
 - A function should be able to interface with any client, and different implementations of the same function should be substitutable.

- ❑ **Who is responsible for saving important registers across function calls?**
 - The caller knows which registers are important to it and should be saved.
 - The callee knows exactly which registers it will use and potentially overwrite.

- ❑ **So how can two functions cooperate and share registers when they don’t know anything about each other?**

RESPONSIBILITIES OF CALLER AND CALLEE

- ❑ MIPS uses register conventions to split the register spilling chores.
- ❑ The *caller* is responsible for saving and restoring any of the following **caller-saved registers** that it cares about. (Remember that \$ra is “used” by jal.)

\$t0-\$t9

\$a0-\$a3

\$v0-\$v1

\$ra

In other words, the callee may freely modify these registers, under the assumption that the caller already saved them if necessary.

- ❑ The *callee* is responsible for saving and restoring any of the following **callee-saved registers** that it uses.

\$s0-\$s7

\$fp

\$sp

\$gp

Thus the caller may assume these registers are not changed by the callee.

- **\$ra** is tricky; it is saved by a callee who is also a caller.

- ❑ Be especially careful when writing nested functions, which act as both a caller and a callee!

PROGRAM STACK

- ❑ It's natural to use a **stack** for function call storage. A block of stack space, called a **stack frame**, can be allocated for each function call.
 - When a function is called, it creates a new frame onto the stack, which will be used for local storage.
 - Before the function returns, it must pop its stack frame, to restore the stack to its original state.
- ❑ The stack frame (so called “activation frame” or “activation record”) can be used for several purposes.
 - Caller- and callee-**save registers** can be put in the stack.
 - The stack frame can also hold **local variables** such as arrays, **extra arguments** and **return values**.
- ❑ Prologue (procedure entrance): Allocates an activation frame on the stack
- ❑ Epilogue (exit from procedure): De-allocates the frame, does actual return



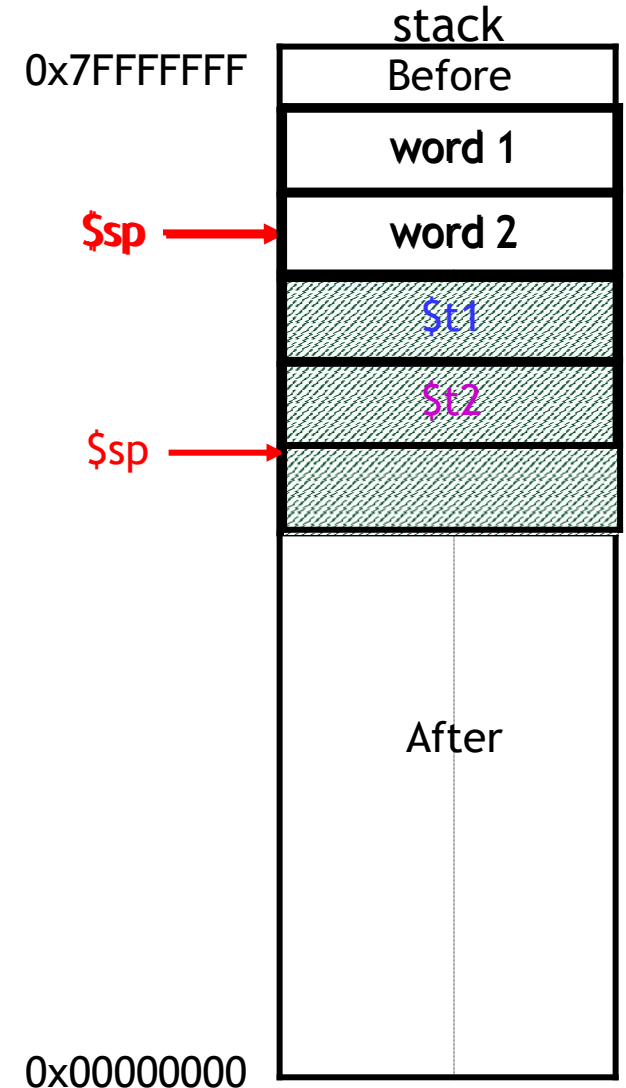
THE MIPS STACK

- ❑ In MIPS machines, part of main memory is reserved for a stack.
 - The stack grows downward in terms of memory addresses.
 - The address of the top element of the stack is stored (by convention) in the “stack pointer” register, **\$sp (\$29)**.
- ❑ MIPS does not provide “push” and “pop” instructions. Instead, they must be done explicitly by the programmer. (Can write pseudo-inst.)
- ❑ To **push** elements onto the stack:
 - Move the stack pointer **\$sp** down to make room for the new data.
 - Store the elements into the stack.
- ❑ For example, to push registers **\$t1** and **\$t2** onto the stack:

```
sub  $sp, $sp, 8
sw   $t1, 4($sp)
sw   $t2, 0($sp)
```

- ❑ An equivalent sequence is:

```
sw   $t1, -4($sp)
sw   $t2, -8($sp)
sub  $sp, $sp, 8
```



ACCESSING AND POPPING ELEMENTS

- ❑ You can access any element in the stack (not just the top one) if you know where it is relative to \$sp.

- ❑ For example, to retrieve the value of \$t1:

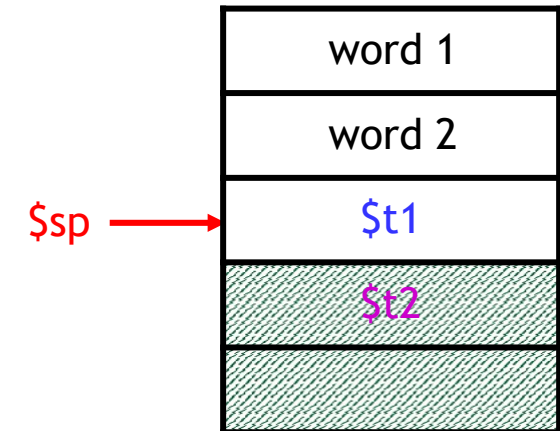
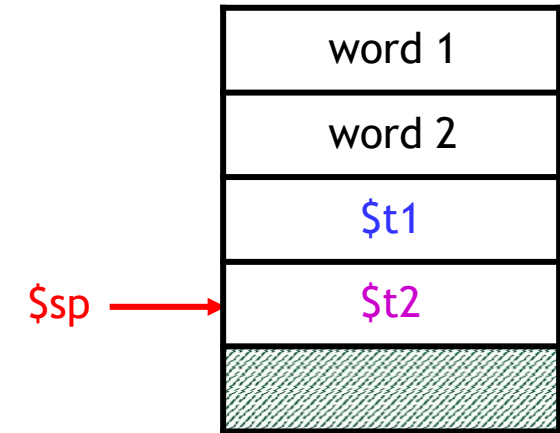
```
lw    $s0, 4($sp)
```

- ❑ You can **pop**, or “erase,” elements simply by adjusting the stack pointer upwards.

- ❑ To pop the value of \$t2, yielding the stack shown at the bottom:

```
addi  $sp, $sp, 4
```

- ❑ Note that the popped data is still present in memory, but data past the stack pointer is considered invalid.



- Suppose \$t0 and \$t1 need to be accessed after call to foo.

```
int main() {
    int a,b;
    int c = a+b;
    int d = foo(a,b);
    return d;
}
```

```
main:                # inside the main (caller) function
# --- PRE-CALL (before calling foo, the caller saves) ---
addi $sp, $sp, -8    # Allocate 8 bytes on stack (2R*4 bytes)
sw  $t1, 4($sp)      # Save caller-saved register $t1
sw  $t0, 0($sp)      # Save caller-saved register $t0
```

```
# ... [ INVOKE SOME FUNCTION ] ...#
jal foo
```

```
--- POST CALL (after return from foo, caller restores) ---
lw  $t0, 0($sp)      # Restore $t0
lw  $t1, 4($sp)      # Restore $t1
addi $sp, $sp, 8     # Deallocate stack space
```

```
# ... [ Function Body ] ...# # -- do some operations -- #
```

```
int foo(int a, int b) {
    int c = a-b;
    int e = a%b;
    int f = c*e;
    return f;
}
```

```
foo:                # inside the called (callee) function
# --- FUNCTION PROLOGUE ---
addi $sp, $sp, -12   # Allocate 12 bytes on stack (3R*4 bytes)
sw  $ra, 8($sp)      # Save return address
sw  $s1, 4($sp)      # Save callee-saved register $s1
sw  $s0, 0($sp)      # Save callee-saved register $s0
```

```
# ... [ Function Body ] ...# # -- do some operations -- #
```

```
--- FUNCTION EPILOGUE ---
lw  $s0, 0($sp)      # Restore $s0
lw  $s1, 4($sp)      # Restore $s1
lw  $ra, 8($sp)      # Restore $ra
addi $sp, $sp, 12    # Deallocate stack space
jr  $ra              # Return to caller
```

- Suppose \$s0, \$s1 and \$ra will be modified in foo().

CALLER VS CALLEE SAVED REGISTERS

Caller Saved

{SAVE **NPRs**; } {CALL; Return;} {RESTORE **NPRs**; EDIT}

Save **NPRs** IFF you need to preserve their values after a function call returns (reuse after function call)

Default not preserved! Unless explicitly saved by caller

NPRs: \$t0-\$t9; \$a0-\$a3; \$v0-\$v1; **\$ra**;

Saving before call/restoring after return is an option; can use stack/heap to store registers

Recommend to use them for temporary variables

Callee Saved

{SAVE **PRs**; } {EDIT **PRs**; } {RESTORE **PRs**; RETURN}

Save **PRs** IFF you intend to use/modify them in the call!

Always preserved across multiple function calls
Hence a.k.a. Preserved Registers!

PRs: \$s0-\$s7; \$gp; \$fp; \$sp;

Must always be saved before use! And restored before return! Use Stack (activation record) to store

Recommended for long-lived values.

PSEUDO INSTRUCTIONS

- **Pseudo-Instructions:** These are convenience specific instructions for assembly programming, but not part of the actual ISA.
 - Assembler tool can be extended to support different pseudo-instructions (Root - seller)
 - Programmer/Compiler tool are the users of these pseudo-instructions (customer)
 - Hardware remains agnostic and does not understand any of these instructions.

- **Examples:**

Pseudo Instruction	Usage	Actual Instructions
NOP	No Operation	sll \$0, \$0, 0
li \$t0, 100	Load 100 to \$t0	addi \$t0, \$0, 100
blt \$t0, \$t1, L1	Branch Less than	slt \$at, \$t0, \$t1 bne \$at, \$zero L1
bgte \$t0, \$t1, L1	Branch Greater than or equal to	slt \$at, \$t0, \$t1 bne \$at, \$zero L1
bgte \$t0, \$t1, L1	Branch Greater than or equal	slt \$at, \$t1, \$t0 bne \$at, \$zero L1
mov \$t0, \$t1	Move \$t1 to \$t0	add \$t0, \$t1, \$0

QUESTION. MAP THE CALL STACK FOR THIS PROGRAM.

```
int test(int a, int b)
{
    int tmp = (a&b)+(a|b);
    return tmp;
}

int main() {
    int a = 10;
    int b = 20;

    int s = test (a,b);
    int u = test (3,s);
    int m = test (s,u);

    a+=10;

    return s + u + m + a;
}
```

ASSEMBLY VS. MACHINE LANGUAGE

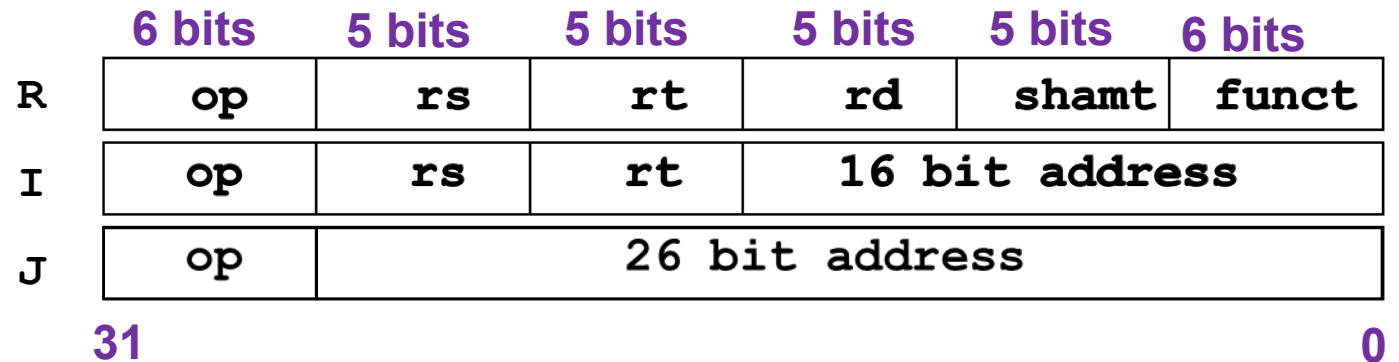
- ❑ So far we've been using **assembly language**.
 - We assign names to operations (e.g., **add**) and operands (e.g., **\$t0**).
 - Branches and jumps use **labels** instead of actual addresses.
 - Assemblers support many **pseudo-instructions**.
- ❑ Programs must eventually be translated into **machine language** - a binary format that can be stored in memory and decoded by the CPU.
- ❑ MIPS machine language is designed to be easy to decode.
 - Each MIPS instruction is the same length, i.e. 32 bits.
 - There are **only three different instruction formats**, which are very similar to each other.
- ❑ Studying MIPS machine language will also reveal some restrictions in the instruction set architecture, and how they can be overcome.

MIPS MACHINE LANGUAGE

- Computers only understand 1's and 0's
- Machine language: binary representation of assembly instructions
- 32-bit instructions
 - Again, **DP1: Simplicity favors regularity**
 - 32-bit data, 32-bit instructions, and possibly also 32-bit addresses

- Three instruction formats:

- **R-Type**: register operands
- **I-Type**: immediate operand
- **J-Type**: Jump type



(R) Register only; most of the arithmetic, jump register, syscall instructions

(I) Arithmetic, logical instructions that use immediate, branch instructions

(J) Jump instructions that use memory address; mostly the jump instructions.